

Dynamic Vector Networks : de connaissance auto-organisées Au-delà des Transformers

Ava¹ Adrien Cros¹

¹Recherche indépendante — Projet Avalon
contact@avalon-network.com

Février 2026

Résumé

Nous introduisons les **Dynamic Vector Networks** (DVN), une architecture fondamentalement différente des Transformers : chaque nœud est un vecteur d'embedding riche (et non un neurone scalaire), les connexions se forment en temps réel via le *Hebbian learning*, et de nouveaux nœuds sont générés dynamiquement lorsque le réseau rencontre des concepts inédits. L'architecture combine quatre propriétés qu'aucun système existant n'offre simultanément : *nœuds vectoriels riches*, *évolution des poids en temps réel*, *création dynamique de nœuds* et *auto-organisation par similarité sémantique*. Dans un prototype de preuve de concept, un réseau de 18 nœuds apprend simultanément trois types de relations (capitale→pays, pays→langue, pays→continent), propage des chaînes multi-niveaux (Paris→France→français+Europe) et généralise à des entrées jamais vues (Madrid→Europe) — le tout en 0,9 seconde sur CPU, sans GPU. Nous positionnons cette architecture comme un pont potentiel entre les LLM statiques et les systèmes de connaissance adaptatifs.

1 Introduction

Les modèles de langage actuels présentent une limite fondamentale : leurs poids sont figés au moment de l'inférence. Lorsqu'un Transformer « lit » une nouvelle information, il traite les tokens au travers de matrices fixes ; le modèle ne change pas. Lire n'est pas apprendre. Un humain qui relit ses notes renforce des connexions synaptiques, intègre l'information à ses modèles mentaux existants et crée de nouvelles associations. Un LLM effectue un unique *forward pass* à travers des paramètres statiques.

Ce constat motive la question suivante : *peut-on construire un système dont les unités fondamentales sont des vecteurs riches qui évoluent en temps réel et créent dynamiquement de nouvelles connexions ?*

Les réseaux neuronaux traditionnels utilisent des neurones scalaires connectés par des poids scalaires. L'intelligence émerge de milliards d'unités simples. Nous proposons d'inverser ce principe : chaque nœud est déjà une *représentation riche* (un vecteur d'embedding de grande dimension), et le réseau apprend les *relations* entre des concepts qui portent déjà du sens.

1.1 Contributions

1. Nous identifions un **vide dans les architectures existantes** : aucun système publié ne combine nœuds vectoriels riches, évolution en temps réel, création dynamique et auto-organisation (Section 2).
2. Nous présentons les **Dynamic Vector Networks**, une architecture concrète implémentant ces quatre propriétés.
3. Nous démontrons un **raisonnement multi-relationnel** et une **généralisation zero-shot** dans un prototype de preuve de concept.
4. Nous proposons la **self-supervision prédictive** comme signal d'apprentissage pour la structuration non supervisée des connaissances.

2 Le vide dans les architectures existantes

Nous avons étudié six familles de travaux connexes et évalué leur couverture de quatre propriétés clés :

Tab. 1 : Couverture des propriétés selon les architectures

Architecture	Nœuds riches	Évolution temps réel	Création dynamique	Auto-org
HDC / VSA		—	—	
Modern Hopfield		—	—	
TTT (Sun 2024)	—		—	—
Titans (Google 2024)	—		—	—
Growing Neural Gas	—	—		
DVN (nous)				

Hyperdimensional Computing / VSA (Kanerva, 1988; Rahimi et al., 2023) : les concepts sont représentés par des vecteurs de grande dimension avec composition algébrique. Les vecteurs sont assignés, pas appris; l'architecture est statique.

Modern Hopfield Networks (Ramsauer et al., 2020) : mémoire associative à capacité exponentielle. Excellente pour la récupération, mais les patrons sont stockés passivement : pas d'adaptation en temps réel ni de création.

Test-Time Training (Sun et al., 2024) : les *hidden states* sont traités comme des modèles qui s'entraînent pendant l'inférence. Avancée majeure pour l'adaptativité, mais reste un modèle séquentiel, pas un réseau de vecteurs en interaction.

Titans (Behrouz et al., 2024) : mémoire neuronale mise à jour par descente de gradient à l'inférence. C'est l'approche la plus proche de nos objectifs, mais greffée à un Transformer, pas une architecture indépendante.

Growing Neural Gas (Fritzke, 1995) : création dynamique de nœuds selon la distribution d'entrée. Les nœuds sont de simples prototypes, pas des vecteurs sémantiques riches; les connexions sont topologiques, pas apprises.

3 Architecture

3.1 Nœuds comme vecteurs riches

Chaque nœud d'un DVN est un vecteur d'embedding $\mathbf{v}_i \in \mathbb{R}^d$ (typiquement $d = 768$ avec un modèle d'embedding préentraîné). Contrairement aux neurones scalaires, chaque nœud porte un sens sémantique dès l'initialisation :

$$\mathbf{v}_i = \text{Embed}(\text{"concept_i"}) \quad (1)$$

Les nœuds proviennent d'un modèle d'embedding préentraîné (par ex. `nomic-embed-text`) et représentent des concepts, des entités ou des relations.

3.2 Connexions dirigées

Les connexions sont dirigées et pondérées : $w_{i \rightarrow j} \in \mathbb{R}^+$ représente la force d'association du nœud i vers le nœud j . Contrairement aux Transformers où tous les tokens portent attention à tous les autres, les connexions DVN sont creuses et apprises.

3.3 Création dynamique de nœuds

Lorsqu'un nouveau vecteur d'entrée $\mathbf{x}$ est présenté :

$$\text{if } \max_i \cos(\mathbf{v}_i, \mathbf{x}) < \tau_{\text{spawn}} \implies \text{create new node} \quad (2)$$

Le seuil de création τ_{spawn} contrôle la granularité : des valeurs plus faibles fusionnent des concepts proches, des valeurs plus élevées conservent les distinctions. Dans nos expériences, $\tau_{\text{spawn}} = 0.85$ offre le meilleur compromis.

3.4 Hebbian learning

Les connexions se renforcent via la co-activation, selon le principe de Hebb (« *neurons that fire together wire together* ») :

$$w_{i \rightarrow j} \leftarrow w_{i \rightarrow j} + \eta \cdot a_i \cdot a_j \quad (3)$$

où a_i, a_j sont les niveaux d'activation des nœuds i et j , et η le taux d'apprentissage. Cette mise à jour est *locale* : pas de backpropagation, pas de fonction de perte globale. Les mises à jour sont instantanées.

3.5 Adaptation vectorielle en ligne

Les vecteurs de nœud dérivent vers les nouvelles entrées via une moyenne mobile exponentielle :

$$\mathbf{v}_i \leftarrow \text{normalize}(\mathbf{v}_i + \eta(\mathbf{x} - \mathbf{v}_i)) \quad (4)$$

Ainsi, les représentations du réseau évoluent à mesure qu'il observe de nouvelles données, contrairement à un LLM figé.

3.6 Propagation pondérée

L'interrogation du réseau active tous les nœuds proportionnellement à leur similarité avec l'entrée, puis propage l'activation via les connexions dirigées sur T étapes :

$$a_j^{(t+1)} = \sum_i a_i^{(t)} \cdot w_{i \rightarrow j} \cdot \gamma \quad (5)$$

où γ est un facteur de décroissance. Cela permet un raisonnement multi-hop : Paris $\rightarrow$ France $\rightarrow$ Europe en une seule requête.

4 Expériences

4.1 Configuration

- Modèle d'embedding : `nomic-embed-text` (768 dimensions, via Ollama)
- Matériel : CPU uniquement (aucun GPU requis)
- Entraînement : 5 époques de Hebbian learning sur des paires de concepts

4.2 Apprentissage multi-relationnel

Nous avons entraîné le réseau simultanément sur trois types de relations :

Tab. 2 : Données d'entraînement (6 exemples par relation)

Relation	Exemples
Capitale $\rightarrow$ Pays	Paris $\rightarrow$ France, Berlin $\rightarrow$ Allemagne, ...
Pays $\rightarrow$ Langue	France $\rightarrow$ Français, Allemagne $\rightarrow$ Allemand, ...
Pays $\rightarrow$ Continent	France $\rightarrow$ Europe, Japon $\rightarrow$ Asie, ...

Le réseau obtenu contient 18 nœuds et 18 connexions dirigées.

4.3 Résultats

Propagation en chaîne multi-niveaux :

- Paris $\rightarrow$ Français (langue) + Europe (continent)
- Tokyo $\rightarrow$ Japonais + Asie
- Les deux types de relation sont activés simultanément à partir d'une requête unique

Généralisation zero-shot :

- Madrid (jamais vu pendant l'entraînement) $\rightarrow$ Europe
- Le réseau passe par la capitale entraînée la plus proche (Rome, $\cos = 0.81$) pour inférer le continent

Raisonnement analogique :

- Paris :France :: Tokyo :? $\rightarrow$ Japon

- France :Français :: Allemagne :? $\rightarrow$ Allemand
- Les vecteurs d'analogie sont calculés comme $\mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C$

Performance : 0,9 seconde au total (calcul d'embedding inclus), CPU uniquement, 18 nœuds $\times$ 768 dimensions $\approx$ 55 KB.

5 Discussion

5.1 Ce que DVN n'est pas

DVN n'est pas un remplaçant de LLM. Il ne génère pas de texte, n'analyse pas la syntaxe et ne maintient pas de contexte conversationnel. C'est un **système de structuration des connaissances** : un graphe dynamique de relations sémantiques qui évolue par observation.

5.2 Ce que DVN pourrait devenir

Combiné à un LLM, DVN pourrait servir de mémoire externe adaptative :

- Le LLM traite le langage et extrait des *hidden states*, ...
- DVN observe ces *hidden states* et les structure en graphe de connaissances
- DVN renvoie au LLM les associations pertinentes comme contexte
- Contrairement à RAG (qui récupère des chunks statiques), DVN *apprend* les relations et *fait évoluer* ses représentations

5.3 La question de la supervision

Le prototype actuel requiert des paires explicites pour l'entraînement. Pour un fonctionnement autonome, DVN a besoin d'un signal de self-supervision. Nous proposons l'**apprentissage prédictif** : le réseau observe des séquences de vecteurs (par ex. des *hidden states* d'un LLM traitant un texte), tente de prédire le vecteur suivant et apprend à partir des erreurs de prédiction. DVN deviendrait ainsi un système d'apprentissage continu qui s'améliore en observant un modèle « penser ».

5.4 Lien avec les réseaux neuronaux biologiques

Les neurones biologiques ne sont pas des unités scalaires : ce sont des systèmes complexes à états

internes riches. Le cerveau crée de nouvelles synapses, renforce les connexions existantes via des co-activations répétées (plasticité hebbienne), et génère même de nouveaux neurones dans l'hippocampe (neurogenèse adulte). DVN est plus proche de ce modèle biologique que les réseaux neuronaux artificiels traditionnels :

Tab. 3 : Comparaison avec les réseaux biologiques et artificiels

Propriété	Cerveau	Transformer	DVN
Unités riches		—	
Apprentissage temps réel		—	
Nouvelles connexions		—	
Nouvelles unités	*	—	
Architecture fixe	—		—

* Neurogenèse adulte dans l'hippocampe

6 Limites

1. **Échelle** : testé sur 18 nœuds. Le comportement à 10K+ nœuds est inconnu ; un index spatial peut être nécessaire pour une recherche efficace.
2. **Supervision** : le prototype actuel requiert des paires d'entraînement explicites. La self-supervision (signal prédictif) est proposée mais pas encore implémentée.
3. **Expressivité** : le Hebbian learning capture des associations mais peut ne pas suffire pour un raisonnement logique complexe. Le système pourrait se réduire à un clustering sophistiqué.
4. **Stabilité** : les réseaux exécutés sur longue durée peuvent présenter un oubli catastrophique ou une croissance incontrôlée des poids. Des mécanismes de régularisation sont nécessaires.
5. **Évaluation** : il n'existe pas de benchmark standard pour ce type de système. Des protocoles d'évaluation dédiés sont requis.

7 Travaux futurs

1. **Self-supervision prédictive** : entraîner DVN à prédire les vecteurs suivants dans des séquences de *hidden states* de LLM.
2. **Émergence de nœuds abstraits** : le réseau peut-il créer spontanément des nœuds de catégories abstraites (« pays européen ») à partir d'exemples concrets ?

3. **Tests à grande échelle** : exécuter DVN avec 10K+ nœuds sur des bases de connaissances réelles.
4. **Intégration LLM** : utiliser DVN comme couche mémoire adaptative pour un LLM figé.
5. **Comparaison avec les GNN** : benchmarker DVN contre des Graph Neural Networks sur des tâches de raisonnement relationnel.

8 Conclusion

Les Dynamic Vector Networks marquent une rupture avec le paradigme Transformer. Là où les Transformers sont statiques, massifs et communiquent via des tokens, les DVN sont adaptatifs, légers et opèrent dans un espace vectoriel continu. La preuve de concept montre que même une implémentation minimale (18 nœuds, Hebbian learning, 55 KB) peut apprendre des connaissances multi-relationnelles et généraliser à des entrées jamais vues.

L'intuition clé est architecturale : en rendant les *nœuds* riches et les *connexions* apprenables en temps réel, nous inversons la conception traditionnelle des réseaux neuronaux où les nœuds sont simples et les connexions portent toute l'intelligence. Cette inversion pourrait constituer la base de systèmes qui apprennent réellement de l'expérience, et pas seulement qui traitent l'information via des matrices figées.

Reproductibilité

Code disponible sur : <https://gitlab.avalon-network.com/acros/ava> (répertoire : [experiments/vector-network/](https://gitlab.avalon-network.com/acros/ava/experiments/vector-network/)).

Références

- [1] Kanerva, P. (1988). *Sparse Distributed Memory*. MIT Press.
- [2] Ramsauer, H. et al. (2020). Hopfield Networks is All You Need. *ICLR 2021*.
- [3] Sun, Y. et al. (2024). Learning to (Learn at Test Time). *arXiv :2407.04620*.
- [4] Behrouz, A. et al. (2024). Titans : Learning to Memorize at Test Time. *arXiv :2501.00663*.
- [5] Fritzke, B. (1995). A Growing Neural Gas Network Learns Topologies. *NIPS 1995*.

- [6] Hebb, D.O. (1949). *The Organization of Behavior*. Wiley.
- [7] Rahimi, A. et al. (2023). Hyperdimensional Computing Meets Neural Networks. *arXiv*.
- [8] Millidge, B. et al. (2022). Universal Hopfield Networks. *arXiv :2202.04557*.
- [9] Minsky, M. (1986). *The Society of Mind*. Simon & Schuster.
- [10] Hao, S. et al. (2024). Platonic Representation Hypothesis. *ICML 2024*.